WEEK 1 · DAY 4 · OVERVIEW

Impurity and Permutation Importance



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Day 3's forest ranks the validation accounts well, and a credit committee asks the next question: which columns does it lean on?

Course 4's Day 9 read two importances off one pruned tree, and they disagreed.

Before the desk quotes either for a forest, it tests them on a column whose worth is known in advance.

It plants a column of random numbers beside the 19 real ones and asks each measure to find it.

The accounts are real, from Taiwan in 2005, so they describe consumer credit there and then, not US credit today.

The four person columns are never features, so no importance is ever read for them.

OBJECTIVES

Rank a forest's columns by impurity and by permutation importance, and plant a column of random numbers to test each measure.

Take impurity importance apart split by split, shuffle related columns one at a time and as a group, and compare shuffles on training and validation rows.

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

Your Route Through Day 4

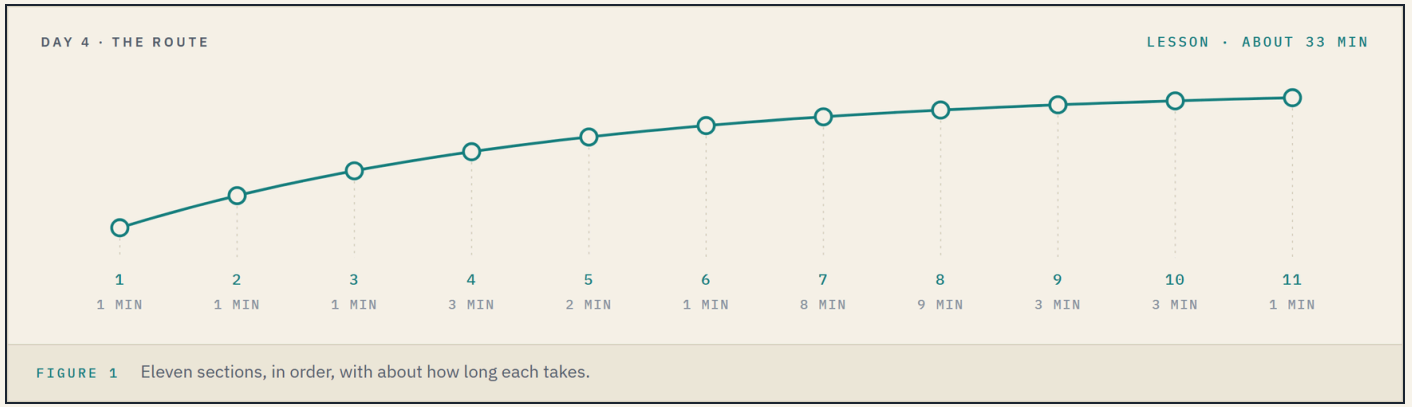


FIGURE 1 Eleven sections, in order, with about how long each takes.

	SECTION	ABOUT
1	Setting Up	1 min
2	Loading the Data	1 min
3	Data Overview	1 min
4	Data Preparation for Modeling	3 min
5	Model Building	2 min
6	Impurity Importance	1 min

	SECTION	ABOUT
7	Permutation Importance	8 min
8	Counting the Splits	9 min
9	Correlated Columns Share Importance	3 min
10	Training Rows Against Validation Rows	3 min
11	Insights and Conclusion	1 min

Time. About 33 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
```

Read the card file and pick 19 columns.

Loading the Data

THE FILE

Data: `credit_card_default.csv`, 30,000 card accounts in Taiwan in 2005: **real**, UCI Machine Learning Repository dataset 350, CC BY 4.0. Citation: Yeh, I. (2009). Default of Credit Card Clients [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C55S3H.

READING THE CARD FILE

```
card = pd.read_csv(DATA + "credit_card_default.csv")
TARGET = "default payment next month"
```

CHOOSING THE FEATURES

```
X = card.drop(columns=["ID", "SEX", "EDUCATION", "MARRIAGE", "AGE", TARGET])
y = card[TARGET]
print(len(X.columns), "features")
```

```
19 features
```

Count the rows and the defaults.

Data Overview

THE FILE'S COLUMNS

- **LIMIT_BAL**: credit limit, NT dollars
- **PAY_0**, **PAY_2** to **PAY_6**: repayment status, September back to April 2005, where -1 is paid on time, 1 to 9 is months late, and -2 and 0 are undocumented (used as numbers, a modelling choice, as in Course 4)
- **BILL_AMT1** to **BILL_AMT6**: the bill, NT dollars, September back to April
- **PAY_AMT1** to **PAY_AMT6**: the amount paid, NT dollars, September back to April
- **default payment next month**: 1 if the account defaulted the following month, **the target**
- **ID**, **SEX**, **EDUCATION**, **MARRIAGE**, **AGE**: never used
- **noise**: added today, a random number for every account, unrelated to anything in the file

ROWS, COLUMNS, AND DEFAULTS

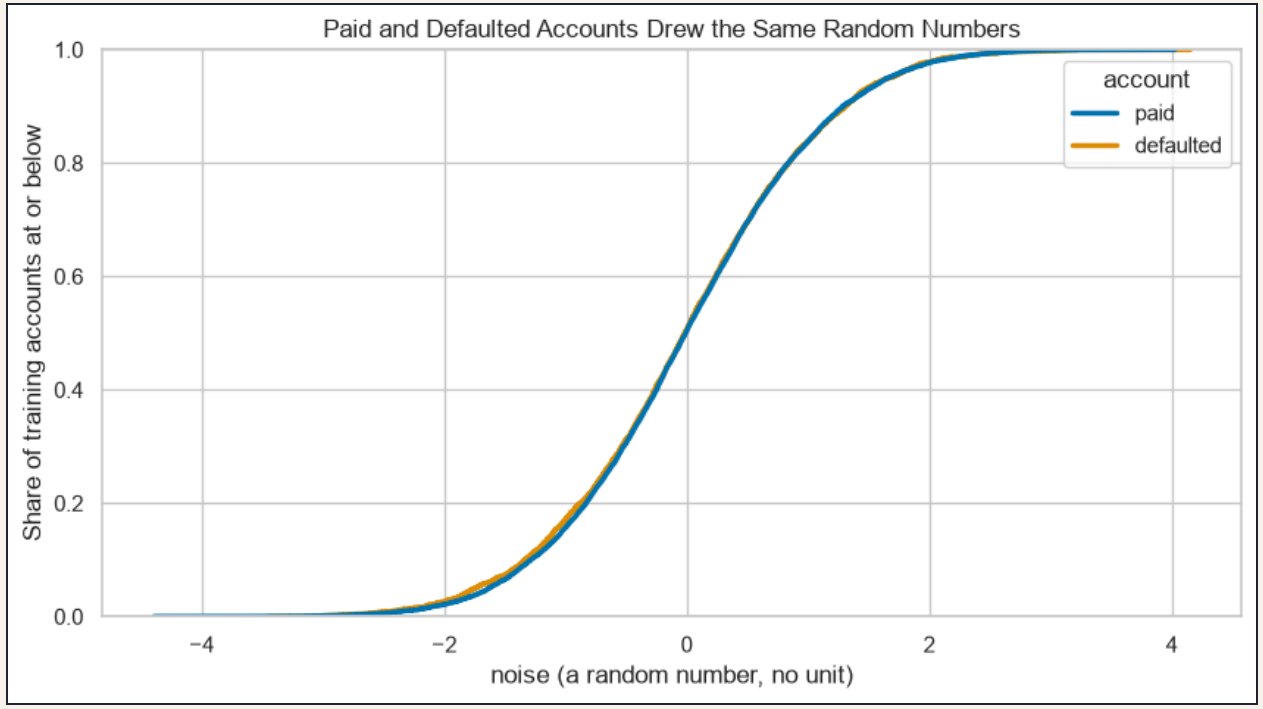
```
print(card.shape)
```

```
(30000, 25)
```

```
Defaults: 6636  Share: 22.12 percent
```

The file Course 4 used, unchanged, so we split it the same way.

Data Preparation for Modeling



COURSE 4'S SPLIT

```
Training: 18000  Validation: 6000  Test: 6000
```

PLANTING THE COLUMN

```
rng = np.random.default_rng(SEED)
X_train["noise"] = rng.normal(size=len(X_train))
X_valid["noise"] = rng.normal(size=len(X_valid))
print(len(COLUMNS), "columns, the last one:", COLUMNS[-1])
```

```
20 columns, the last one: noise
```

The planted column knows nothing about default, so a measure worth trusting puts it at the bottom.

Fit 200 full trees on 20 columns.

Model Building

DAY 3'S DEFAULT FOREST

```
forest = RandomForestClassifier(n_estimators=200, random_state=SEED, n_jobs=1).fit(X_train, y_train)
valid_output = forest.predict_proba(X_valid)[:, 1]
fitted_auc = roc_auc_score(y_valid, valid_output)
print("Validation ROC AUC:", round(fitted_auc, 4))
print("Training ROC AUC:", round(roc_auc_score(y_train, forest.predict_proba(X_train)[:, 1]), 4))
```

```
Validation ROC AUC: 0.7667
Training ROC AUC: 1.0
```

HOW BIG THE TREES GREW

```
Leaves per tree: 2507
Depth per tree: 43
```

A strong forest on unseen accounts. Next: the columns it leans on.

Rank the columns by the forest's Gini share.

Impurity Importance

THE IDEA

Course 4's Day 9 defined it: each column's share of all the Gini decrease the tree's questions made on the training rows.

A forest's **`.feature_importances_`** is that share averaged over its 200 trees.

THE QUESTION

Q. Of the 20 columns, where will **`noise`** rank by impurity importance? Write down a number.

THE FOREST'S OWN RANKING

```
impurity = pd.Series(forest.feature_importances_, index=COLUMNS).sort_values(ascending=False)
print("noise ranks", int(impurity.rank(ascending=False)["noise"]), "of 20")
impurity.head(6)
```

Permutation Importance

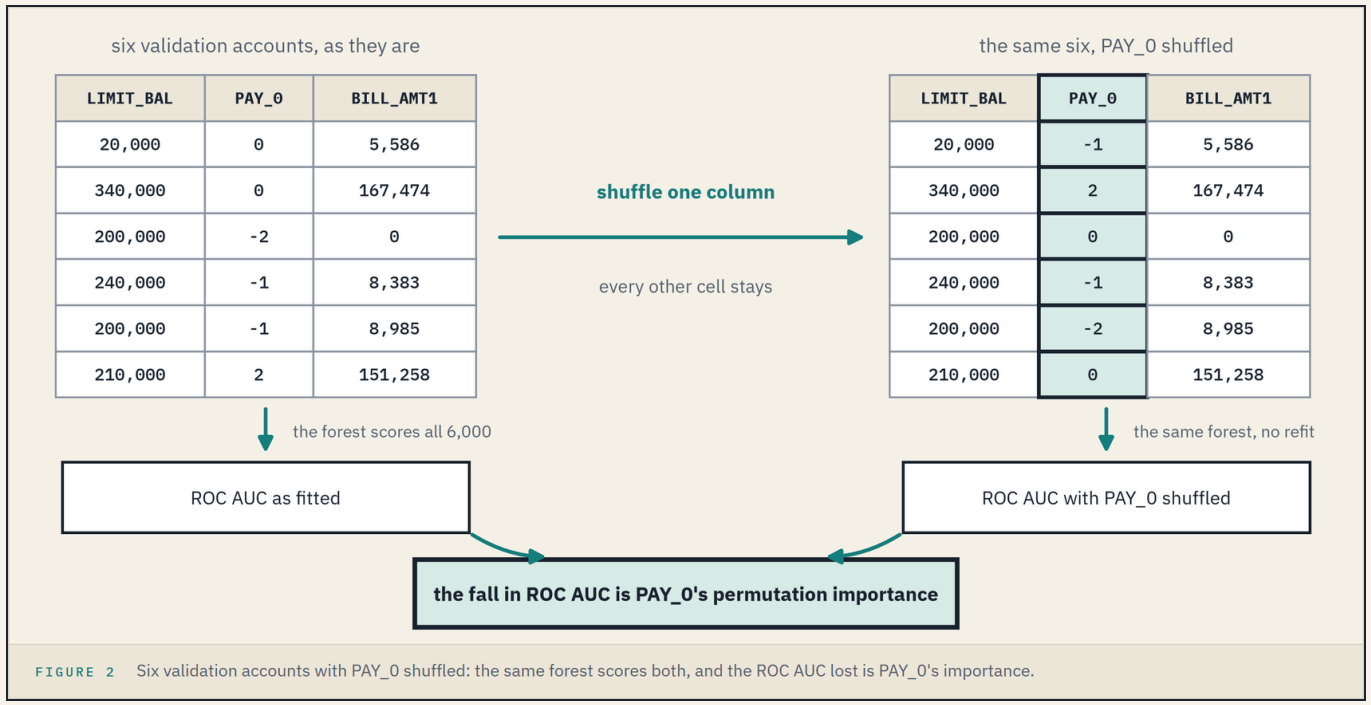


FIGURE 2 Six validation accounts with PAY_0 shuffled: the same forest scores both, and the ROC AUC lost is PAY_0's importance.

THE IDEA

Permutation importance shuffles one column's values among the validation accounts, scores the same forest again, and reads the fall in ROC AUC.

There is no refit.

HOW A COLUMN IS SHUFFLED

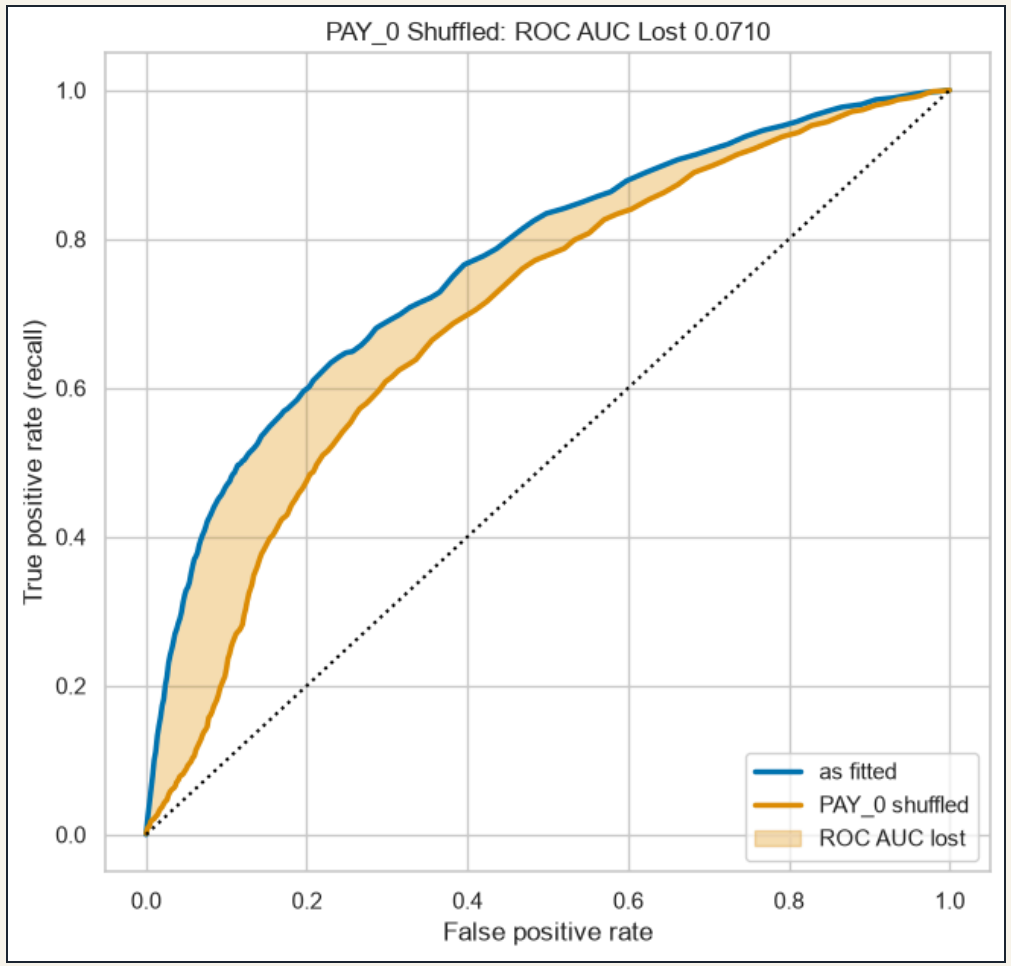
`.sample(frac=1, random_state=SEED)` draws every value once in a random order: a shuffle.

ONE COLUMN SHUFFLED

```
print("As fitted:", round(fitted_auc, 4), " PAY_0 shuffled:", round(roc_auc_score(y_valid, shuffled_output("PAY_0")), 4))
```

```
As fitted: 0.7667  PAY_0 shuffled: 0.6957
```

Shuffling PAY_0



WHAT THE SHADED AREA IS

The shaded area is the ROC AUC lost: what the forest leans on **`PAY_0`** for, on accounts it never saw.

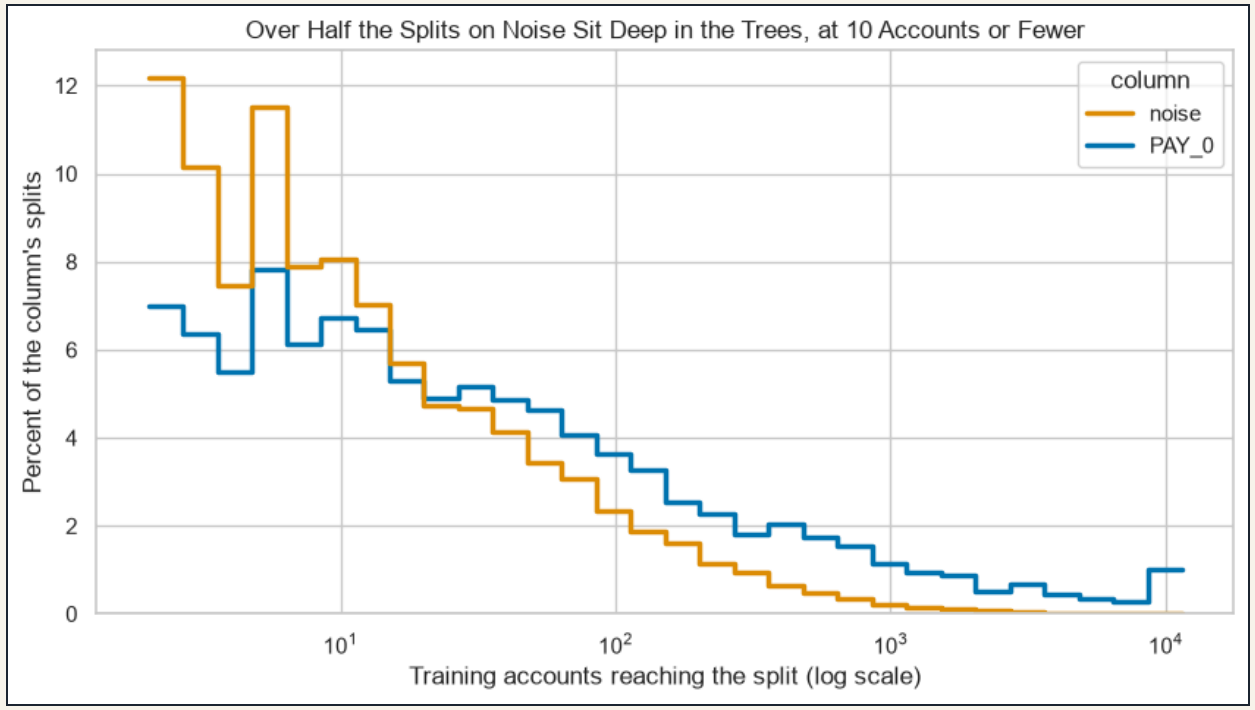
TEN SHUFFLES FOR EVERY COLUMN

`permutation_importance` (Course 4's Day 9) shuffles every column **`n_repeats`** times: **`.importances_mean`** is the mean fall, and **`.importances_std`** its standard deviation over the shuffles.

THE LEADERS

	PERMUTATION	SD
PAY_0	0.0690	0.0040
LIMIT_BAL	0.0284	0.0041
PAY_AMT1	0.0166	0.0027
PAY_2	0.0117	0.0018
PAY_AMT2	0.0081	0.0021

Counting the Splits



COUNTING THE QUESTIONS

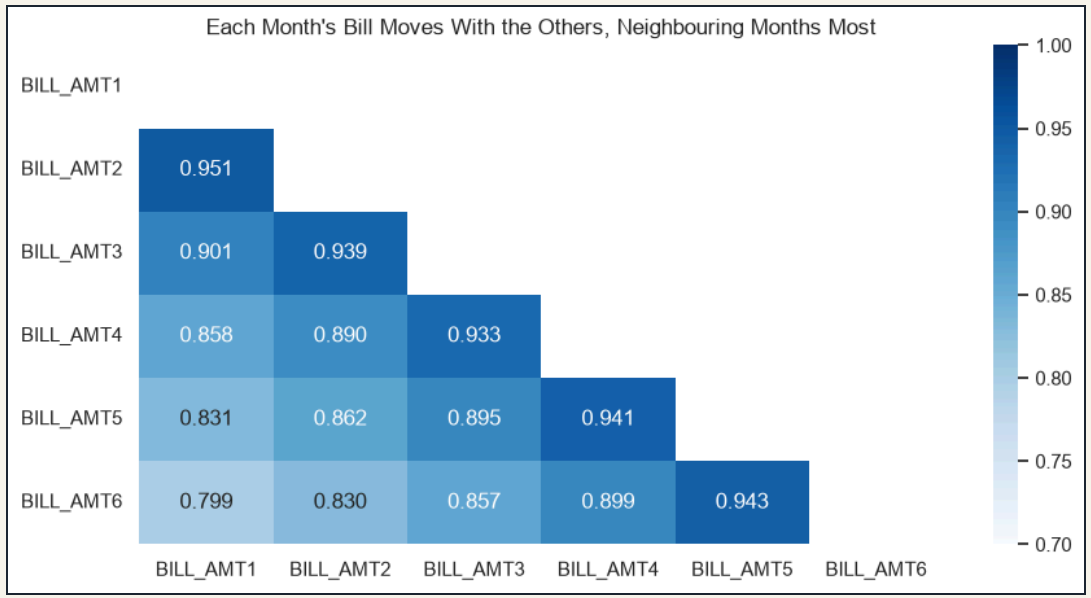
Every node of a tree asks about one column: `.tree_.feature` (Day 1) holds its number, -2 at a leaf.

WHERE THE QUESTIONS ARE ASKED

```
noise  median accounts: 9.0  splits on 10 or fewer: 55 percent
PAY_0  median accounts: 19.0  splits on 10 or fewer: 37 percent
```

A full tree keeps splitting until each leaf is pure, so its last splits separate a few training accounts from each other.

Correlated Columns Share Importance



THE SIX BILLS

The bills correlate at 0.799 to 0.951, neighbouring months highest.

THE QUESTION

Q. Shuffle all six bills together, with one row order for the six. How much validation ROC AUC will the forest lose? Write down a number.

TEN GROUP SHUFFLES

```
group_drops = []
for k in range(10):
    shuffled = X_valid.copy()
    shuffled[bills] = X_valid[bills].iloc[rng.permutation(len(X_valid))].to_numpy()
    group_drops.append(fitted_auc - roc_auc_score(y_valid, forest.predict_proba(shuffled)[:, 1]))
print("Six bills together: mean fall", round(np.mean(group_drops), 4), " sd", round(np.std(group_drops), 4))
```

Training Rows Against Validation Rows

THE QUESTION

The forest scores 1.0 on its own training rows.

Q. Shuffle on 6,000 of the training rows instead of the validation rows. Where will **noise** rank by permutation importance? Write down a number.

THE SAME TEN SHUFFLES

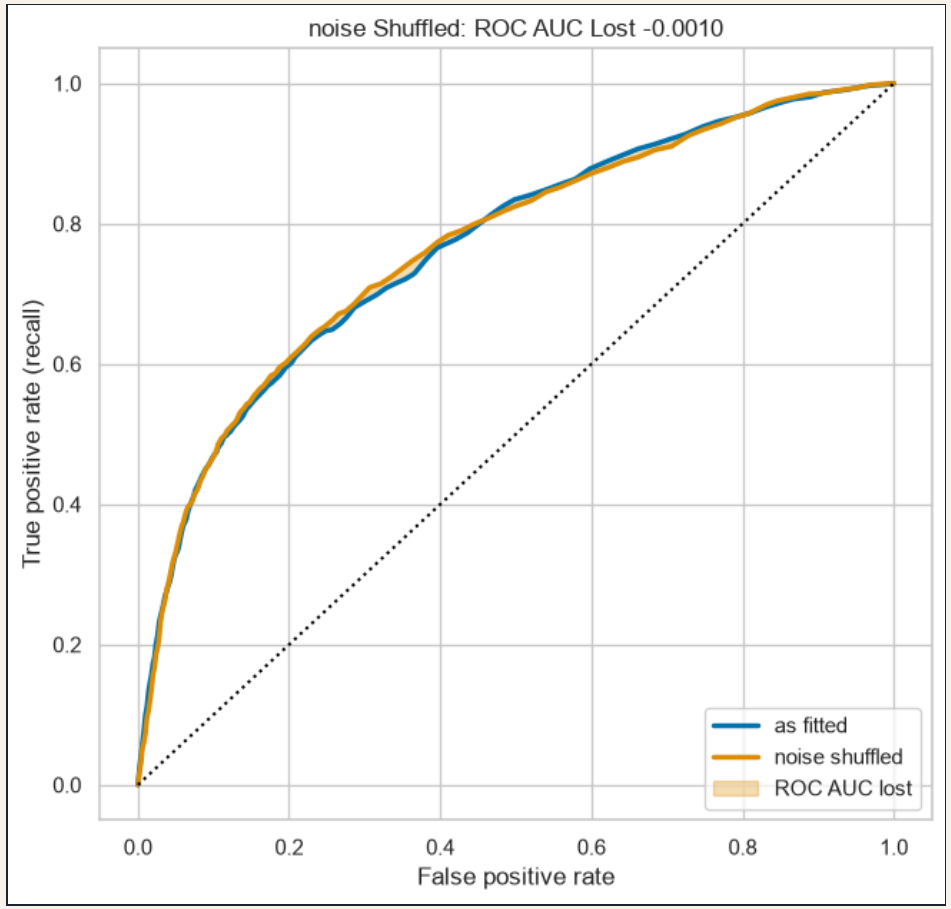
```
train_perm = permutation_importance(forest, X_train.iloc[:6000], y_train.iloc[:6000], scoring="roc_auc", n_repeats=10,
                                    random_state=SEED, n_jobs=1)
```

FOUR VIEWS OF ONE FOREST

```
views = pd.DataFrame({"impurity, full trees": impurity, "impurity, 20-account leaves": leaf_impurity[20],
                      "permutation, validation": importance["permutation"],
                      "permutation, training": importance["training"]})
ranks = views.rank(ascending=False).astype(int).sort_values("permutation, validation")
```

Ask the question on rows the model has never seen, with a column of random numbers beside the real ones.

Insights and Conclusion



THE NOISE BAND

```
band = 2 * importance.loc["noise", "sd"]
print("Noise band: plus or minus", round(band, 4))
print("Above it:", list(importance.index[importance["permutation"] > band]))
```

```
Noise band: plus or minus 0.0042
Above it: ['PAY_0', 'LIMIT_BAL', 'PAY_AMT1', 'PAY_2', 'PAY_AMT2', 'PAY_3', 'PAY_4']
```

READING THE PICTURE

With **noise** shuffled the two curves lie on top of each other, and ROC AUC "falls" by -0.0010: no area at all.

Plant a column of random numbers, shuffle on unseen rows, and shuffle related columns together.

You Can Now

- Read a forest's impurity importance and measure its permutation importance on validation rows.
- Plant a column of random numbers and read every importance against its noise band.
- Count a forest's splits by column and the accounts that reach each split.
- Shuffle related columns one at a time and as a group.
- Compare permutation importance on training and validation rows.

NEXT

WEEK 1 · DAY 5

Case Study: Bagging and the Spread Cross-Section

Day 5 takes forests to the bond spreads Course 4 fitted by least squares.